

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 4
4. SECURE COMMUNICATIONS 4
5. SECONDARY TESTING 4
6. FUNCTIONAL TESTING 5
7. SUMMARY 5
8. INDUSTRY STANDARD BEST PRACTICES 6

Document Revision History

Version	Date	Author	Comments
1.0	[Date]	Adalberto Flores	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Adalberto Flores

1. Algorithm Cipher

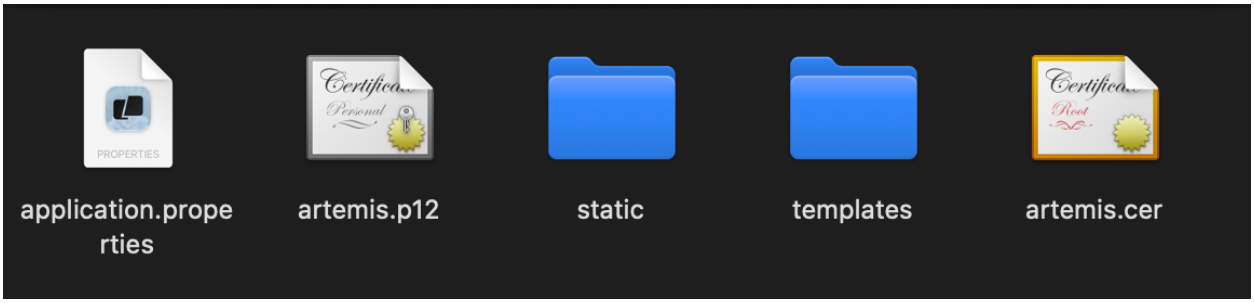
I picked SHA-256 because Artemis Financial needs a reliable way to make sure client financial data insta altered during transfer. SHA-256 is a one-way cryptographic hash that takes any input string and converts it into a fixed 256-bit (64-character hex) digest. It runs in 512-bit blocks through 64 rounds of math operations, offering 128 bits of collision resistance. That means finding two different inputs that produce the exact same has is computationally impossible.

Unlike AES or RSA, a hash function doesn't use keys. It is deterministic, so the exact same input will always generate the exact same checksum. However, cryptographically secure random numbers are still essential in the background when generating certificates and setting up TLS handshakes so attackers can't guess session keys.

Older algorithms like MD% and SHA-1 were once standard, but both were broken by practical collision attacks where different file generated matching hashes. Modern standards set by NIST require SHA-2 or SHA-3, making SHA-256 the industry baseline for verifying financial transactions and certificates today.

2. Certificate Generation

Insert a screenshot below of the CER file.



3. Deploy Cipher

Insert a screenshot below of the checksum verification.

Name: Adalberto Flores | Data: Artemis Financial Secure Checksum Verification | SHA-256: 6b082f2ae77499facdce96889e7e5db6d92a911bd03e7031810b682b6a43b7b9

4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.



5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis perform provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project: ssl-server

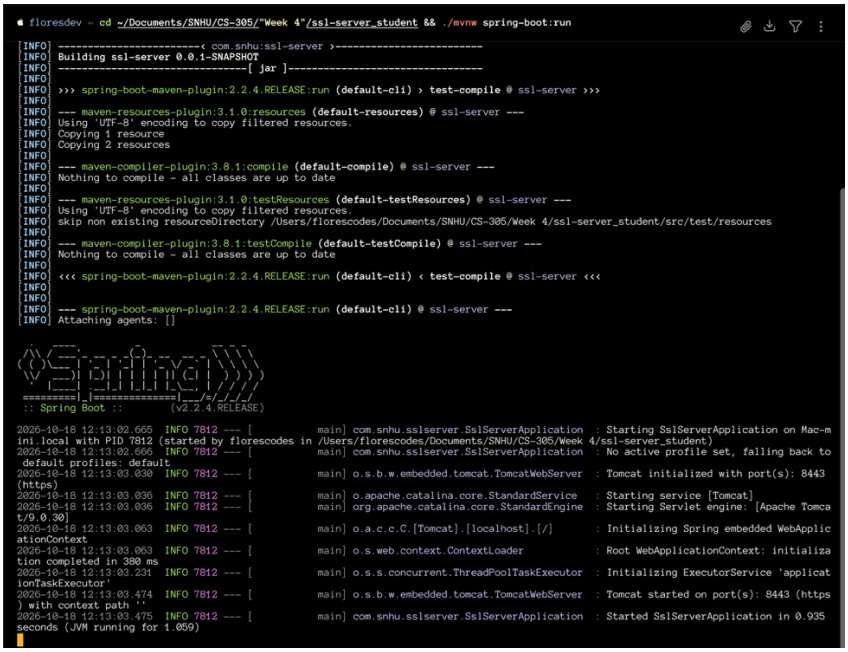
com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 10.0.4
- *Report Generated On*: Sun, 27 Sep 2026 14:57:44 -0500
- *Dependencies Scanned*: 49 (31 unique)
- *Vulnerable Dependencies*: 16
- *Vulnerabilities Found*: 295
- *Vulnerabilities Suppressed*: 0
- ...

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



```
floresdev ~ cd ~/Documents/SNHU/CS-305/"Week 4"/ssl-server_student && ./mvnw spring-boot:run
[INFO] ----------------------< com.snhu:ssl-server >-----------------------
[INFO] Building ssl-server 0.0.1-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] >>> spring-boot-maven-plugin:2.2.4.RELEASE:run (default-cli) > test-compile @ ssl-server >>>
[INFO]
[INFO] --- maven-resources-plugin:3.1.0:resources (default-resources) @ ssl-server ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 1 resource
[INFO] Copying 2 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ ssl-server ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:3.1.0:testResources (default-testResources) @ ssl-server ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /Users/florescodes/Documents/SNHU/CS-305/Week 4/ssl-server_student/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ ssl-server ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] <<< spring-boot-maven-plugin:2.2.4.RELEASE:run (default-cli) < test-compile @ ssl-server <<<
[INFO]
[INFO]
[INFO] --- spring-boot-maven-plugin:2.2.4.RELEASE:run (default-cli) @ ssl-server ---
[INFO] Attaching agents: []

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.4.RELEASE)

2026-10-18 12:13:02.665  INFO 7812 --- [           main] com.snhu.sslserver.SslServerApplication  : Starting SslServerApplication on Mac-m
ini.local with PID 7812 (started by florescodes in /Users/florescodes/Documents/SNHU/CS-305/Week 4/ssl-server_student)
2026-10-18 12:13:02.666  INFO 7812 --- [           main] com.snhu.sslserver.SslServerApplication  : No active profile set, falling back to
 default profiles: default
2026-10-18 12:13:03.030  INFO 7812 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8443
(https)
2026-10-18 12:13:03.036  INFO 7812 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-10-18 12:13:03.036  INFO 7812 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomca
t/9.0.30]
2026-10-18 12:13:03.063  INFO 7812 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplic
ationContext
2026-10-18 12:13:03.063  INFO 7812 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initializa
tion completed in 380 ms
2026-10-18 12:13:03.231  INFO 7812 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicat
ionTaskExecutor'
2026-10-18 12:13:03.474  INFO 7812 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8443 (https
) with context path ''
2026-10-18 12:13:03.475  INFO 7812 --- [           main] com.snhu.sslserver.SslServerApplication  : Started SslServerApplication in 0.935
seconds (JVM running for 1.059)
```

7. Summary

To secure Artemis Financial, I worked on the key areas outlined in the vulnerability assessment process flow diagram:

- Network Security: I switched the web server from open HTTP on port 8080 over to HTTPS on port 8443 so traffic can't be sniffed.
- Cryptography/ Data in Transit: I created a PKCS12 keystore (artemis.p12), generated our self-signed certificate, and configured Spring Boots aplication.properties to enforce TLS encryption.

- Data Integrity: I refactored the controller using Java's MessageDigest with SHA-256 to create a checksum verification endpoint(/hash)
- Code & Dependencies: I ran OWSAP Dependency-Check to make sure none of our Maven dependencies brought in known security flaws.

Adding Layers of Security:

Instead of relying on just one fix, I added multiple layers:

- Transport Layer: HTTPS/TLS keeps anyone on the network from reading the traffic in plain text.
- Application Layer: The SHA-256 checksum makes sure that even if data moves across systems, we can verify it wasn't modified or corrupted.
- Build Layer: Static scanning with OWASP lets us catch vulnerable third-party libraries before deploying code to production.

8. Industry Standard Best Practices

- Use Trusted Libraries: I used Javas built-in java.security.MessageDigest rather than trying to write a custom hashing method.
- Secure Defaults: I disabled plaintext HTTP and configured the server to enforce HTTPS over port 8443 using a PKCS12 keystore.
- Automated Auditing: I used OWASP Dependency-Check during the Maven build to catch vulnerable third-party libraries before deployment.

For a consulting firm handling investments, retirements and banking records, security directly protects clients trust and the business. Preventing data tampering avoids regulatory fines (like GLBA or PCI compliance violations), stops unauthorized data modifications, and prevents costly security remediation down the road.